Compiler Design
Week 3

Adi Ramanarayan
230905380 48
Batch A2

Lab Exercises:

Q1. Write functions to identify the following tokens.

a. Arithmetic, relational and logical operators.

b. Special symbols, keywords, numerical constants, string literals and identifiers.

Program (Header file for final lexer, Contains previous weeks preprocessor functions):

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <ctype.h>
#include "preprocessor.h"

#define MAX_KEYWORDS 32
#define MAX_TOKEN_LEN 64

typedef enum {
    TOKEN_KEYWORD,
    TOKEN_IDENTIFIER,
    TOKEN_NUMBER,
    TOKEN_STRING,
    TOKEN_CHAR,
    TOKEN_OPERATOR,
    TOKEN_SPECIAL_SYMBOL,
    TOKEN_EOF,
```

```
    TOKEN_ERROR
} TokenType;

typedef struct {
    TokenType type;
    char lexeme[MAX_TOKEN_LEN];
} Token;

typedef struct {
    int has_pushback;
    int pushback_char;
} CharBuffer;

/* Standard C89/C90 Keywords */
const char *keywords[MAX_KEYWORDS] = {
    "auto", "break", "case", "char", "const", "continue", "default",
    "do", "double", "else", "enum", "extern", "float", "for",
    "goto", "if", "int", "long", "register", "return",
    "short", "signed", "sizeof", "static", "struct",
    "switch", "typedef", "union", "unsigned",
    "void", "volatile", "while"
};

/* Initialize the lookahead buffer */
void initBuffer(CharBuffer *cb) {
    cb->has_pushback = 0;
    cb->pushback_char = 0;
}

/* Initialize the preprocessing state */
void initPreprocessState(PreprocessState *st)
{
    st->in_string = 0;
    st->in_char = 0;
    st->at_line_start = 1;
    st->in_directive = 0;
    st->prev_was_space = 0;
    st->line = 1;
    st->column = 0;
}

/* Push a character back into the custom buffer */
void pushBack(CharBuffer *cb, int ch) {
    if (ch == EOF) return;
```

```
    if (cb->has_pushback) {
        fprintf(stderr, "Error: CharBuffer overflow (double
pushback)\n");
        exit(1);
    }

    cb->pushback_char = ch;
    cb->has_pushback = 1;
}

/* Get next char from buffer (if available) or preprocessor */
int nextChar(FILE *src, PreprocessState *st, CharBuffer *cb) {
    if (cb->has_pushback) {
        cb->has_pushback = 0;
        return cb->pushback_char;
    }
    return getCleanChar(src, st);
}

/* Linear search for keyword match */
int is_keyword(const char *token) {
    for (int i = 0; i < MAX_KEYWORDS; i++) {
        if (strcmp(token, keywords[i]) == 0)
            return 1;
    }
    return 0;
}

Token getNextToken(FILE *src, PreprocessState *st, CharBuffer *cb) {
    Token tok;
    int ch;
    int idx = 0;

    ch = nextChar(src, st, cb);

    /* --- End of File --- */
    if (ch == EOF) {
        tok.type = TOKEN_EOF;
        strcpy(tok.lexeme, "EOF");
        return tok;
    }

    /* --- Identifier / Keyword --- */
    if (isalpha(ch) || ch == '_') {
        tok.lexeme[idx++] = ch;
```

```
        while ((ch = nextChar(src, st, cb)) != EOF && (isalnum(ch) ||
ch == '_')) {
            if (idx >= MAX_TOKEN_LEN - 1) {
                tok.type = TOKEN_ERROR;
                strcpy(tok.lexeme, "TOKEN_TOO_LONG");
                return tok;
            }
            tok.lexeme[idx++] = ch;
        }

        // Push back the non-identifier char so we process it next
time
        if (ch != EOF) pushBack(cb, ch);

        tok.lexeme[idx] = '\0';
        tok.type = is_keyword(tok.lexeme) ? TOKEN_KEYWORD :
TOKEN_IDENTIFIER;
        return tok;
    }

    /* --- Numeric Constants --- */
    if (isdigit(ch)) {
        tok.type = TOKEN_NUMBER;
        tok.lexeme[idx++] = ch;

        // Greedy match for hex (x/a-f) and floats (.)
        while ((ch = nextChar(src, st, cb)) != EOF &&
               (isdigit(ch) || ch == '.' || ch == 'x' || ch == 'X' ||
               (ch >= 'a' && ch <= 'f') || (ch >= 'A' && ch <= 'F')))
{

            if (idx >= MAX_TOKEN_LEN - 1) {
                tok.type = TOKEN_ERROR;
                strcpy(tok.lexeme, "TOKEN_TOO_LONG");
                return tok;
            }
            tok.lexeme[idx++] = ch;
        }

        if (ch != EOF) pushBack(cb, ch);
        tok.lexeme[idx] = '\0';
        return tok;
    }
```

```
/* --- String Literals --- */
if (ch == '"') {
    tok.type = TOKEN_STRING;
    tok.lexeme[idx++] = ch;

    while ((ch = nextChar(src, st, cb)) != EOF) {
        if (idx >= MAX_TOKEN_LEN - 1) break;

        // C strings cannot contain unescaped newlines
        if (ch == '\n') {
            tok.type = TOKEN_ERROR;
            strcpy(tok.lexeme, "UNTERMINATED_STRING");
            return tok;
        }

        tok.lexeme[idx++] = ch;

        // Handle Escape Sequences (e.g. \")
        if (ch == '\\') {
            tok.lexeme[idx++] = nextChar(src, st, cb);
            continue;
        }

        if (ch == '"') break; // End of string
    }

    // Check valid termination
    if (tok.lexeme[idx-1] != '"') {
        tok.type = TOKEN_ERROR;
        strcpy(tok.lexeme, "UNTERMINATED_STRING");
        return tok;
    }

    tok.lexeme[idx] = '\0';
    return tok;
}

/* --- Character Constants --- */
if (ch == '\'') {
    tok.type = TOKEN_CHAR;
    tok.lexeme[idx++] = ch;

    while ((ch = nextChar(src, st, cb)) != EOF) {
        if (idx >= MAX_TOKEN_LEN - 1) break;
```

```
            if (ch == '\n') {
                tok.type = TOKEN_ERROR;
                strcpy(tok.lexeme, "UNTERMINATED_CHAR");
                return tok;
            }

            tok.lexeme[idx++] = ch;
            if (ch == '\'') break;
        }

        if (tok.lexeme[idx-1] != '\'') {
            tok.type = TOKEN_ERROR;
            strcpy(tok.lexeme, "UNTERMINATED_CHAR");
            return tok;
        }

        tok.lexeme[idx] = '\0';
        return tok;
    }

    /* --- Operators --- */
    // Check if char is in the set of operator start characters
    if (strchr("+-*/%=!<>|&", ch)) {
        tok.type = TOKEN_OPERATOR;
        tok.lexeme[idx++] = ch;

        // Lookahead to handle composite operators (==, !=, <=, etc)
        int la = nextChar(src, st, cb);

        switch (ch) {
            case '+':
                if (la == '+' || la == '=') tok.lexeme[idx++] = la;
                else pushBack(cb, la);
                break;

            case '-':
                if (la == '-' || la == '=' || la == '>')
tok.lexeme[idx++] = la;
                else pushBack(cb, la);
                break;

            case '=': case '!': case '<': case '>':
                // Matches ==, !=, <=, >=, <<, >>
                if (la == '=' || la == ch) tok.lexeme[idx++] = la;
                else pushBack(cb, la);
```

```
                break;

            case '&': case '|':
                // Matches &&, ||
                if (la == ch) tok.lexeme[idx++] = la;
                else pushBack(cb, la);
                break;

            default: // *, /, %
                if (la == '=') tok.lexeme[idx++] = la;
                else pushBack(cb, la);
                break;
        }

        tok.lexeme[idx] = '\0';
        return tok;
    }

    /* --- Special Symbols --- */
    if (strchr("(){}[];,:.", ch)) {
        tok.type = TOKEN_SPECIAL_SYMBOL;
        tok.lexeme[0] = ch;
        tok.lexeme[1] = '\0';
        return tok;
    }

    /* --- Unknown Token --- */
    tok.type = TOKEN_ERROR;
    tok.lexeme[0] = ch;
    tok.lexeme[1] = '\0';
    return tok;
}
```

Q2. Design a lexical analyzer that includes a getNextToken() function for processing a simple C program. The analyzer should construct a token structure containing the row number, column number, and token type for each identified token. The getNextToken() function must ignore tokens located within singleline or multi-line comments, as well as those found inside string literals. Additionally, it should strip out preprocessor directives.

Program:

```
/*Lexical analyzer to tokenize simple C programs. Takes filepath as
input and outputs symbol table
to a file.*/
#include "identifier_functions.h"
#include <stdio.h>
#include <stdlib.h>

int main(){
    FILE *sourceFile = fopen("input.c", "r");
    if (sourceFile == NULL) {
        perror("Error opening file");
        return EXIT_FAILURE;
    }

    PreprocessState preprocessState;
    initPreprocessState(&preprocessState);
    CharBuffer charBuffer;
    initBuffer(&charBuffer);

    Token token;
    do {
        token = getNextToken(sourceFile, &preprocessState,
&charBuffer);
        if(token.type == TOKEN_ERROR) continue;
        printf("Token Type: %d, Lexeme: %s\n", token.type,
token.lexeme);
    } while (token.type != TOKEN_EOF);

    fclose(sourceFile);
    return EXIT_SUCCESS;
}
```

Program (sample input file):

```
//Test file for lexical analyzer
```

```c
#include <stdio.h>
#include <stdlib.h>

int main() {
    printf("Hello, World!\n");
    int a,b;
    a = 5;
    b = a + 10;
    for(int i=0; i< b; i++){
        printf("%d\n", i);
    }
    return 0;
}
```

Output:

```
./la
Token Type: 1, Lexeme: main
Token Type: 6, Lexeme: (
Token Type: 6, Lexeme: )
Token Type: 6, Lexeme: {
Token Type: 1, Lexeme: printf
Token Type: 6, Lexeme: (
Token Type: 3, Lexeme: "Hello, World!"
Token Type: 6, Lexeme: )
Token Type: 6, Lexeme: ;
Token Type: 0, Lexeme: int
Token Type: 1, Lexeme: a
Token Type: 6, Lexeme: ,
Token Type: 1, Lexeme: b
Token Type: 6, Lexeme: ;
Token Type: 1, Lexeme: a
Token Type: 5, Lexeme: =
Token Type: 2, Lexeme: 5
Token Type: 6, Lexeme: ;
Token Type: 1, Lexeme: b
Token Type: 5, Lexeme: =
Token Type: 1, Lexeme: a
Token Type: 5, Lexeme: +
Token Type: 2, Lexeme: 10
Token Type: 6, Lexeme: ;
Token Type: 0, Lexeme: for
Token Type: 6, Lexeme: (
Token Type: 0, Lexeme: int
Token Type: 1, Lexeme: i
Token Type: 5, Lexeme: =
Token Type: 2, Lexeme: 0
Token Type: 6, Lexeme: ;
Token Type: 1, Lexeme: i
Token Type: 5, Lexeme: <
Token Type: 1, Lexeme: b
Token Type: 6, Lexeme: ;
Token Type: 1, Lexeme: i
Token Type: 5, Lexeme: ++
Token Type: 6, Lexeme: )
Token Type: 6, Lexeme: {
Token Type: 1, Lexeme: printf
Token Type: 6, Lexeme: (
Token Type: 3, Lexeme: "%d\n"
Token Type: 6, Lexeme: ,
Token Type: 1, Lexeme: i
Token Type: 6, Lexeme: )
Token Type: 6, Lexeme: ;
Token Type: 6, Lexeme: }
Token Type: 0, Lexeme: return
Token Type: 2, Lexeme: 0
Token Type: 6, Lexeme: ;
Token Type: 6, Lexeme: }
Token Type: 7, Lexeme: EOF
```